Lab 1
Nikolaos Karapatsias 10661
Evangelos Stergiopoulos ????

This document provides a detailed showoff of a hybrid C/Arm Assembly project based on stm32f4 processor implementing a complex string hash system.

We are starting off with the c code which acts as a starting point providing all the necessary information for the functions ahead implemented in Arm Assembly language.

```c
#include <stdio.h>

extern int hash_function(char* pin);
extern int clearance_level_function(int hash);
extern int Lucas_sequence_function(int clearance);

int main(void){
  char pin[] = "A382";

  int hashed_pin = hash_function(pin);
  int clearance = clearance_level_function(hashed_pin);
  int sequence_output = Lucas_sequence_function(clearance);

  printf("Pin is: %s", pin);
  printf("\nHashed Pin is: %d", hashed_pin);
  printf("\nClearance is: %d", clearance);
  printf("\nLucas Sequence Output: %d", sequence_output);
  return 0;
}
```

P1 C setup code

Then we are proceeding to create the first function which will implement the basic hash logic. The point is to end up with a final hash integer number. To achieve that we are using the ASCII encoding standard, and based on the type of each char in the string we are calculating the final hash number. Hence, for each capital letter we are multiplying its ascii code by two and adding it to the final sum as shown below. Note that this logic is engulfed in an if statement and only if the char is between the interval A-Z proceed to calculations, otherwise we are making a jump/branch to the next label, for this example check_lowercase, or if the statement is taken we are repeating the loop by jumping back to hash_loop up until NULL \n character appears.

```
cmp r2, #'A'
blt check_lowercase
cmp r2, #'Z'
bgt check_lowercase
// After these cases we know it is uppercase
add r1, r1, r2, lsl #1 // We are multiplying the ASCII code by 2 (left shift by 1) and add it to the hash
b hash_loop
```

P2 Checking for capital letters ~ Arm Assembly

If the previous check is false, then we proceed to the lowercase case in which we are converting the current letter to its corresponding capital one and add it to the final sum. Note that the result of the conversion is calculated using a bitwise AND mask by which we are inverting all the bits of the current lowercase ASCII letter and then computing its AND result with the number 32 which corresponds to binary 100000 leading to the same exact number

but with its 5th bit flipped, which is exactly the difference between the capital and the lowercase form of the same letter. Then we are adding the result to the final sum

```
cmp r2, #'a'
blt check_digits
cmp r2, #'z'
bgt check_digits
bic r2, r2, #32 // This mask converts the ASCII character from lowercase to uppercase
add r1, r1, r2 // Add the new ASCII to the hash
```

P3 Checking for lowercase letters ~ Arm Assembly

Last but not least we are converting the ASCII numbers into integers using the provided lookup table shown below, adding their result to the final vfollowing the same pattern

0->2, 1->3, 2->5, 3->7, 4->11, 5->13, 6->17, 7->19, 8->23, 9->29

P4 Lookup table for the num conversion ~ Arm Assembly

```
cmp r2, #'0'
blt hash_loop
cmp r2, #'9'
bgt hash_loop

sub r2, r2, #'0' // '0' to '9' is 48-57, so we normalize to 0-9
ldr r4, [r3, r2, lsl #2]
// Since a word in the lookup table is 4 bytes and r4 is the base address of the LUT, we are loading from the memory r4 + r2 * 4
add r1, r1, r4
b hash_loop
```

P5 Checking for lowercase letters ~ Arm Assembly

Then we are implementing an assembly function which receives the final hashed integer from the previous step and calculating the appearance of 1 in the binary form of the hashed result, by checking each time the Least Significant Bit (LSB), updating a counter and then right shifting the number to check the next bit up until all bits are checked.

```
// We check the LSB each time by doing a bitwise AND between the hash and 1
tst r0, #1 //Sets the zero flag
it ne
addne r1, r1, #1 //Uses the zero flag to determine whether to increment the counter
// Now we shift the hash by one bit so we can check the next bit, which becomes the LSB
lsr r0, r0, #1
// Once we have proccesed all the ones, cmp r1, #0 will give 0 and we will branch to calculate_mod
b count_ones_loop
```

P6 Searching for ones in the hash result ~ Arm Assembly

Then we are calculating the modulo of the division between the counter and 6 which results to a number between 0 and 5 and then return its result indicating the clearance level

```
calculate_mod:
    cmp r1, #6
    blt done //If the count is less than 6, that is our remainder
    sub r1, r1, #6 //If it is not, subtract 6
    b calculate_mod

done:
    // Store the result in the memory
    ldr r2, =clearance_result
    str r1, [r2]

    // Move the output to r0
    mov r0, r1
    bx lr
```

Finally we are creating a third assembly function which using the clearance level N from the previous stage to calculate the Nth term of Lucas sequence. Note that Lucas sequence is denoted as: L(n) = L(n-1)+L(n-2) for n>1 and L(0) = 2, L(1) = 1. The implementation is shown below:

Firstly we are checking for the two initial case where n is 0 or 1 and loading the corresponding value to the r0 return register.

```
// Base cases
cmp r0, #0
it eq
moveq r0, #2
beq done
cmp r0, #1
it eq
moveq r0, #1
beq done
```

Then, if n is greater than 0 and 1 we implementing the recursive logic. First saving the current value of n i register r4, and calculating the Lucas result of L(n-1) and L(n-2), since we have 2 initial cases. Since this code is expected to be executed multiple times per call we are using the stack to push/store the values of N (in r4) and the result L(n-1) (r5) hence when a child call is finished we know those values calculated in the parent section

```
push {r4, r5, lr}


// Recursive steps
mov r4, r0 // Save current n in r4
// n-1
sub r0, r4, #1
bl Lucas_sequence_function // Calculates L(n-1) and puts it at r0
mov r5, r0 // Save L(n-1) at r5
// n-2
sub r0, r4, #2
bl Lucas_sequence_function // Calculates L(n-2) and puts it at r0
add r0, r0, r5 // L(n-1) + L(n-2)
```